

01

Estacionamento IoT

Protótipo didático com ESP32,
sensor ultrassônico e MQTT

Eduardo Pereira Valias

Vinicius Telles e Silva

Introdução

O Smart Parking IoT é um protótipo que transforma uma vaga comum em uma vaga inteligente, detectando ocupação em tempo real e transmitindo status para um aplicativo móvel.

A demonstração usa um *HC-SR04*, **ESP8266** e comunicação **MQTT**, evidenciando arquitetura completa: sensor, processamento de borda, comunicação e visualização.





02

Visão Geral do Projeto

Problema resolvido

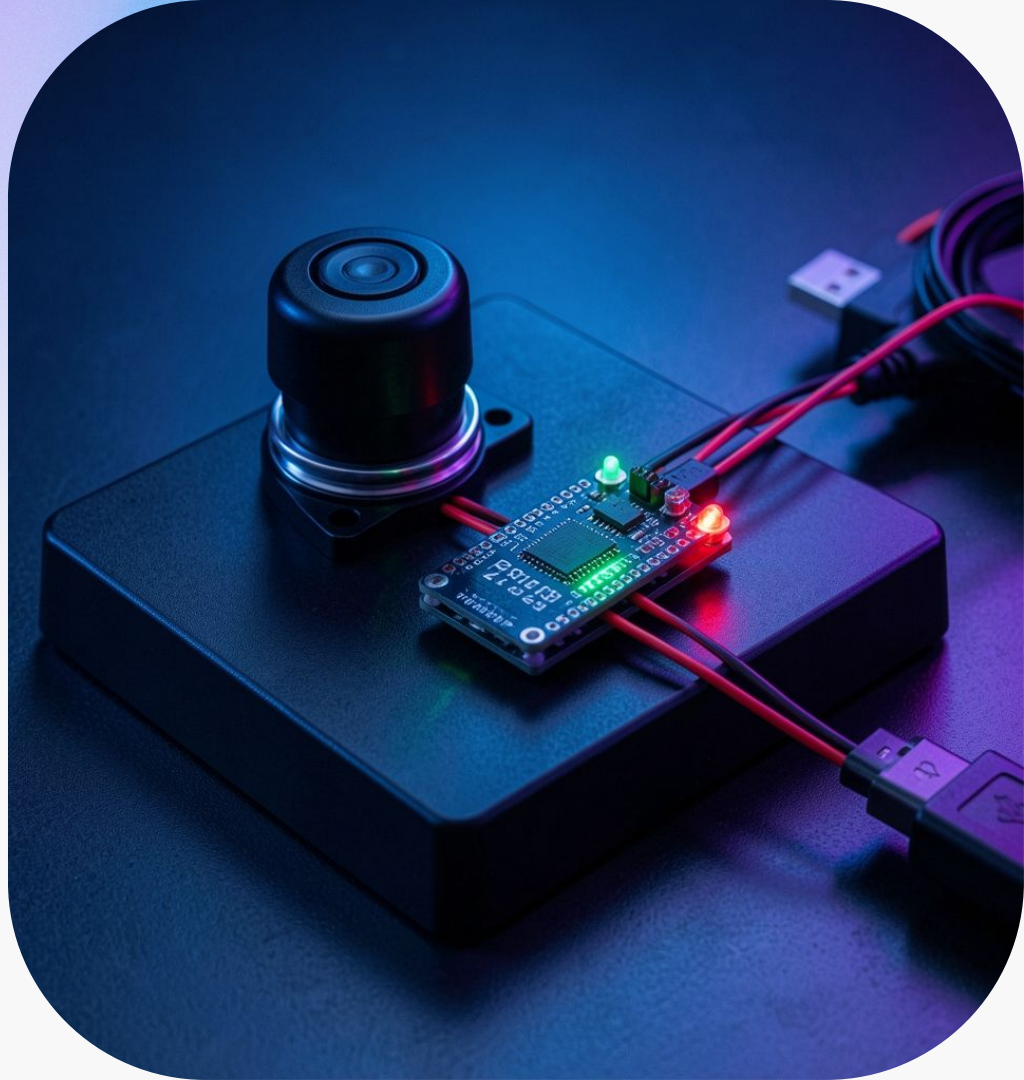
Motoristas perdem tempo procurando vagas e operadores não têm visão em tempo real da ocupação. Isso gera congestionamento interno, inspeção manual e subutilização do espaço.

A solução busca reduzir tempo de procura, melhorar gestão de vagas e fornecer dados para decisões operacionais.

Proposta de solução

Cada vaga usa *sensor ultrassônico* e **ESP8266** para medir distância e publicar status via **MQTT**. LEDs indicam localmente livre/ocupada; app ou dashboard implementa tópicos MQTT para visualização.

Arquitetura escalável: múltiplas vagas, broker MQTT, armazenamento de séries temporais e integração com painéis de entrada, cancelas e sistemas de pagamento.



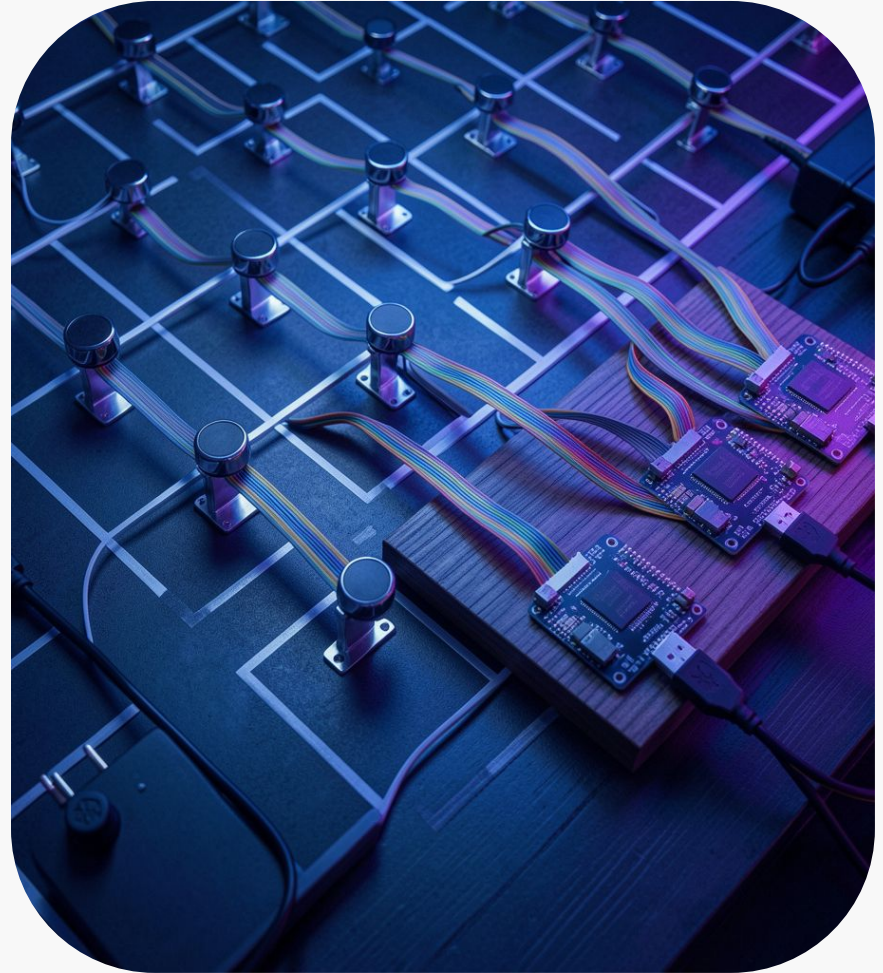
Benefícios e mercado

Reduz busca por vagas e otimiza uso do espaço, gerando economia de tempo para motoristas e operadores.

A solução fornece dados históricos para *decisão operacional* e é escalável para integração com painéis, apps e cobrança. Mercado com forte crescimento projeta adoção em campi, condomínios e operadores privados.

03

Protótipo e Arquitetura



Componentes principais (ESP8266, HC-SR04, LEDs)

Protótipo usa *ESP8266* (Wi-Fi) e sensor *HC-SR04* para medir distância e determinar estado da vaga.

LEDs locais indicam status; componentes escolhidos por baixo custo e facilidade de demonstração. Projeto modular para fácil replicação em maquetes e pilotos.

Componentes principais (ESP8266, HC-SR04, LEDs)

Faz a leitura da distância utilizando o HC-SR04. De acordo com a distância calculada altera o status da vaga e os LEDs acesos

```
bool medirDistancia() {  
    digitalWrite(trigPin, LOW);  
    delayMicroseconds(2);  
  
    digitalWrite(trigPin, HIGH);  
    delayMicroseconds(10);  
    digitalWrite(trigPin, LOW);  
  
    duration = pulseIn(echoPin, HIGH, 30000);  
  
    if (duration == 0) {  
        return false;  
    }  
  
    distanceCm = duration * SOUND_VELOCITY / 2;  
    distanceInch = distanceCm * CM_TO_INCH;  
  
    return true;  
}  
  
if (medirDistancia()) {  
    String statusVaga;  
  
    if (distanceCm <= DISTANCIA_OCUPADA_CM) {  
        statusVaga = "ocupada";  
    }  
    else {  
        statusVaga = "livre";  
    }  
  
    // Se o sistema automatico estiver ligado,  
    // o sensor controla os LEDs normalmente  
    if (sistemaAtivo) {  
        atualizarLedsAutomatico(statusVaga);  
    }  
}
```

Componentes principais (ESP8266, HC-SR04, LEDs)

Faz a leitura da distância utilizando o HC-SR04. De acordo com a distância calculada altera o status da vaga e os LEDs acesos

```
// Funcao para atualizar LEDs automaticamente de acordo com o sensor
void atualizarLedsAutomatico(String statusVaga) {
    if (statusVaga == "ocupada") {
        digitalWrite(ledVerde, LOW);
        digitalWrite(ledVermelho, HIGH);
    }
    else if (statusVaga == "livre") {
        digitalWrite(ledVerde, HIGH);
        digitalWrite(ledVermelho, LOW);
    }
    else {
        digitalWrite(ledVerde, LOW);
        digitalWrite(ledVermelho, LOW);
    }
}
```

Comunicação MQTT e tópicos sugeridos

Dispositivo publica status via **MQTT** para um broker; app e dashboards assinam tópicos.

Exemplos de tópicos:

inatel/smartparking/vaga01/status,
inatel/smartparking/vaga01/comando,
inatel/smartparking/vaga01/resposta.

Payloads em JSON permitem interoperabilidade e automação.



Integração e expansão (dashboards, cancela, pagamentos)

Arquitetura permite integração com *dashboards*, InfluxDB/Grafana, Node-RED, painéis de entrada e sistemas de pagamento.

Escalável para múltiplas vagas, reservas e integração com cancelas e identificação (RFID/câmeras) em implementações comerciais.

Comunicação MQTT

```
// Topico onde o ESP8266 recebe comandos vindos do MQTTX
const char* mqtt_command_topic = "inatel/smartparking/vaga01/comando";

// Topico para o ESP8266 responder qual comando executou
const char* mqtt_response_topic = "inatel/smartparking/vaga01/resposta";
```

```
void reconnect_mqtt() {
  while (!client.connected()) {
    Serial.print("Conectando ao MQTT... ");

    String clientId = "ESP8266-SmartParking-";
    clientId += String(ESP.getChipId(), HEX);

    if (client.connect(clientId.c_str(), mqtt_user, mqtt_password)) {
      Serial.println("conectado!");

      client.publish(
        mqtt_topic,
        "{\"status\": \"ESP8266 conectado ao HiveMQ\"}"
      );

      // Aqui o ESP8266 se inscreve no topico de comando
      client.subscribe(mqtt_command_topic);

      Serial.print("Inscrito no topico de comando: ");
      Serial.println(mqtt_command_topic);

      client.publish(mqtt_response_topic, "ESP8266_ONLINE");
    }
    else {
      Serial.print("falhou, rc=");
      Serial.print(client.state());
      Serial.println(" tentando novamente em 5 segundos...");
      delay(5000);
    }
  }
}
```



Comunicação MQTT

```
// Liga o sistema automatico novamente
if (comando == "SISTEMA_LIGAR") {
  sistemaAtivo = true;

  Serial.println("Sistema automatico ligado.");
  client.publish(mqtt_response_topic, "SISTEMA_LIGADO");

  return;
}

// Desliga o sistema automatico e libera o modo manual
if (comando == "SISTEMA_DESLIGAR") {
  sistemaAtivo = false;

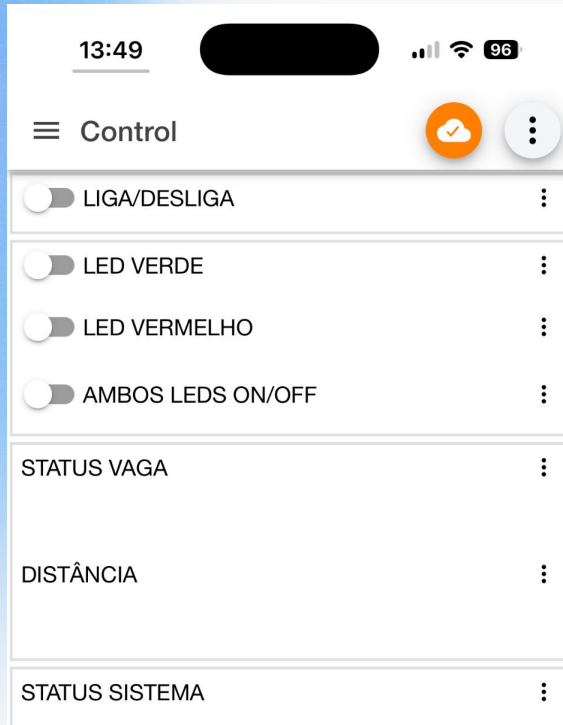
  digitalWrite(ledVerde, LOW);
  digitalWrite(ledVermelho, LOW);

  Serial.println("Sistema automatico desligado. Modo manual liberado.");
  client.publish(mqtt_response_topic, "SISTEMA_DESLIGADO_MODO_MANUAL_ATIVO");

  return;
}
```



Comunicação MQTT



Comunicação MQTT

```
void publicarStatus() {  
  if (medirDistancia()) {  
    String statusVaga;  
  
    if (distanceCm <= DISTANCIA_OCUPADA_CM) {  
      statusVaga = "ocupada";  
    }  
    else {  
      statusVaga = "livre";  
    }  
  
    // Se o sistema automatico estiver ligado,  
    // o sensor controla os LEDs normalmente  
    if (sistemaAtivo) {  
      atualizarLedsAutomatico(statusVaga);  
    }  
  }  
}
```



Comunicação MQTT

```
String payload = "{";
payload += "\"vaga\": \"vaga01\", ";
payload += "\"sistema\": \"\"";
payload += sistemaAtivo ? "ligado" : "desligado";
payload += "\", ";
payload += "\"modo\": \"\"";
payload += sistemaAtivo ? "automatico" : "manual";
payload += "\", ";
payload += "\"status\": \"\"";
payload += statusVaga;
payload += "\", ";
payload += "\"distance_cm\": ";
payload += String(distanceCm, 2);
payload += ", ";
payload += "\"distance_inch\": ";
payload += String(distanceInch, 2);
payload += "}";

Serial.print("Distancia: ");
Serial.print(distanceCm);
Serial.print(" cm | Status: ");
Serial.print(statusVaga);
Serial.print(" | Sistema: ");
Serial.println(sistemaAtivo ? "ligado" : "desligado");

client.publish(mqtt_topic, payload.c_str());
```



Comunicação MQTT

```
else {  
    if (sistemaAtivo) {  
        atualizarLedsAutomatico("erro");  
    }  
  
    Serial.println("Falha ao medir distancia.");  
  
    String payload = "{";  
    payload += "\"vaga\": \"vaga01\",";  
    payload += "\"sistema\": \"\"";  
    payload += sistemaAtivo ? "ligado" : "desligado";  
    payload += "\",";  
    payload += "\"status\": \"erro\",";  
    payload += "\"erro\": \"falha_na_medicao\"";  
    payload += "}";  
  
    client.publish(mqtt_topic, payload.c_str());  
}
```



Comunicação MQTT

```
void loop() {  
  if (!client.connected()) {  
    reconnect_mqtt();  
  }  
  
  client.loop();  
  
  unsigned long now = millis();  
  
  if (now - lastPublish >= publishInterval) {  
    lastPublish = now;  
    publicarStatus();  
  }  
}
```



Conclusões

Protótipo demonstra conceito completo: detecção por ultrassom, processamento em borda com *ESP8266* e comunicação **MQTT**.

Viável como solução de baixo custo para pilotos e ambientes controlados; para mercado, demanda evolução em robustez, encapsulamento e segurança para instalação externa.

Conclusões

ParkHelp oferece sistemas para estacionamentos internos e externos com sensores ultrassônicos, infravermelho e câmeras.

A Bosch possui sensores de estacionamento que identificam ocupação e permitem recursos como busca de vagas, navegação até a vaga e reserva.